

2026-05-05-p1-f09-slash-command-system

P1-F09 — Slash Command System (user-defined Markdown commands) Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (- []) syntax for tracking.

Goal: Add a parallel slash-command surface for user-defined Markdown commands stored in `.helix/commands/*.md` (project) and `~/ .config/helixcode/commands/*.md` (user). YAML frontmatter, regex-based variable substitution (`{{ARG1}}`, `{{ARG.name}}`, `{{SELECTION}}`, `{{CURRENT_FILE}}`, `{{CWD}}`, `{{ENV.NAME}}`, `{{FILE:path}}`), fsnotify watcher hot-reloads. Both `/commands slash` and `helixcode commands cobra` surface inspection + reload + run.

Architecture: Add `internal/commands/markdown_commands.go` (same package as built-ins; `MarkdownCommand` implements `Command`). `MarkdownLoader` scans both dirs (project overrides user), parses frontmatter via `gopkg.in/yaml.v3`, registers each command. `MarkdownWatcher` uses `fsnotify` (already-direct dependency) with 200ms debounce. Single-pass regex `\{\{\{([A-Z_][A-Z0-9_]*(\.[A-Za-z_][A-Za-z0-9_]*)?(:[^\}]+)?)\}\}\}` matches all token forms; `tokenResolver` dispatches per-class.

Tech Stack: Go 1.26, testify v1.11, `gopkg.in/yaml.v3` (in `go.mod`), `github.com/fsnotify/fsnotify` (in `go.mod` from F05 hooks reload), `github.com/spf13/cobra` v1.8 (in `go.mod`). **No new external dependencies.**

Spec: `docs/superpowers/specs/2026-05-05-p1-f09-slash-command-system-design.md` (commit 79e8bd1)

Working directory for go commands: `helix_code/`. Git from meta-repo root.

Anti-bluff smoke (FULL 4-term):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|placeholder" \
  internal/commands/markdown_commands.go internal/commands/
  markdown_watcher.go \
  internal/commands/commands_command.go cmd/cli/commands_cmd.go
&& echo BLUFF || echo clean
```

Task list



P1-F09-T01 — bootstrap evidence + advance PROGRESS



- ☐ P1-F09-T02 — markdown_commands.go: MarkdownCommand type + frontmatter parser + regex substitution (TDD)
 - ☐ P1-F09-T03 — MarkdownLoader: scan both dirs, project overrides user, register/unregister (TDD)
 - ☐ P1-F09-T04 — markdown_watcher.go: fsnotify watcher + 200ms debounce (TDD)
 - ☐ P1-F09-T05 — commands_command.go: /commands slash + cmd/cli/commands_cmd.go cobra (TDD)
 - ☐ P1-F09-T06 — cmd/cli/main.go startup wiring + integration test (real fs + real subprocess)
 - ☐ P1-F09-T07 — Challenge with runtime evidence + cross-compile check
 - ☐ P1-F09-T08 — Feature 9 close-out + push to 4 remotes
-

Task 1: Bootstrap evidence + advance PROGRESS

Files: modify docs/improvements/06_phase_1_evidence.md (append F09 section header) and docs/improvements/PROGRESS.md (replace current focus + add F09 task list block of 8 items after F08's). Commit subject: docs(P1-F09-T01): bootstrap Phase 1 / Feature 9 evidence + advance PROGRESS. Co-Authored-By trailer.

The F09 evidence header:

P1-F09 — Slash Command System

```
**Spec:** `docs/superpowers/specs/2026-05-05-p1-f09-slash-
command-system-design.md` (commit `79e8bd1`)
**Plan:** `docs/superpowers/plans/2026-05-05-p1-f09-slash-
command-system.md`
**Started:** 2026-05-05
**Status:** active
```

Task evidence trail

(filled in commit-by-commit as tasks land)

Current focus block:

Current focus

- ****Active phase:**** P1 — claude-code feature porting
- ****Active feature:**** F09 — Slash Command System
- ****Active task:**** P1-F09-T01 — bootstrap evidence + advance PROGRESS

- ****Last completed:**** P1-F08-T09 — Feature 8 (Plan Mode) close-out + push
- ****Owner:**** agent (Claude Opus 4.7)
- ****Started:**** 2026-05-04
- ****Last touched:**** 2026-05-05
- ****Blocked-on:**** none

F09 task list block (8 items, all unchecked):

Active feature task list (P1-F09: Slash Command System)

- [] P1-F09-T01 — bootstrap evidence + advance PROGRESS
- [] P1-F09-T02 — markdown_commands.go: MarkdownCommand + parser + substitution (TDD)
- [] P1-F09-T03 — MarkdownLoader: scan dirs + register/unregister (TDD)
- [] P1-F09-T04 — markdown_watcher.go: fsnotify + debounce (TDD)
- [] P1-F09-T05 — /commands slash + helixcode commands cobra (TDD)
- [] P1-F09-T06 — main.go wiring + integration test
- [] P1-F09-T07 — Challenge with runtime evidence + cross-compile check
- [] P1-F09-T08 — Feature 9 close-out + push

Task 2: markdown_commands.go — MarkdownCommand + parser + substitution (TDD)

Files: create helix_code/internal/commands/markdown_commands.go, helix_code/internal/commands/markdown_commands_test.go.

Test file (write FAILING test first):

```
package commands

import (
    "context"
    "os"
    "path/filepath"
    "strings"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestParseFrontmatter_Valid(t *testing.T) {
    body := `---
title: Refactor
```

description: Rename a function

variables:

function_name: ""

Body text {{ARG1}}.`

```
cmd, err := parseMarkdownCommand("refactor", body, "/tmp/
    refactor.md")
```

```
require.NoError(t, err)
```

```
assert.Equal(t, "refactor", cmd.Name())
```

```
assert.Equal(t, "Rename a function", cmd.Description())
```

```
assert.Contains(t, cmd.body, "Body text {{ARG1}}.")
```

```
assert.Contains(t, cmd.variables, "function_name")
```

```
}
```

```
func TestParseFrontmatter_NoFrontmatter(t *testing.T) {
```

```
cmd, err := parseMarkdownCommand("plain", "Just a body.", "/
    tmp/plain.md")
```

```
require.NoError(t, err)
```

```
assert.Equal(t, "plain", cmd.Name())
```

```
assert.Equal(t, "Just a body.", cmd.body)
```

```
}
```

```
func TestParseFrontmatter_Malformed(t *testing.T) {
```

```
body := `---
```

```
title: oops
```

```
NOT YAML
```

```
---
```

```
body`
```

```
_, err := parseMarkdownCommand("bad", body, "/tmp/bad.md")
```

```
require.Error(t, err)
```

```
}
```

```
func TestSubstitute_PositionalArgs(t *testing.T) {
```

```
cmd := &MarkdownCommand{name: "x", body: "{{ARG1}} and
    {{ARG2}}"}
```

```
out, err := cmd.render(&CommandContext{Args:
    []string{"hello", "world"}})
```

```
require.NoError(t, err)
```

```
assert.Equal(t, "hello and world", out)
```

```
}
```

```
func TestSubstitute_NamedArg(t *testing.T) {
```

```
cmd := &MarkdownCommand{
```

```
    name: "x",
```

```
    body: "Function: {{ARG.function_name}}",
```

```
    variables: map[string]string{"function_name": "myFunc"},
```

```
}
```

```

    out, err := cmd.render(&CommandContext{Args: nil})
    require.NoError(t, err)
    assert.Equal(t, "Function: myFunc", out)
}

func TestSubstitute_SelectionAndCurrentFile(t *testing.T) {
    cmd := &MarkdownCommand{name: "x", body: "Sel: {{SELECTION}}
        | File: {{CURRENT_FILE}}"}
    out, err := cmd.render(&CommandContext{Selection:
        "the_text", CurrentFile: "main.go"})
    require.NoError(t, err)
    assert.Equal(t, "Sel: the_text | File: main.go", out)
}

func TestSubstitute_CWD(t *testing.T) {
    cmd := &MarkdownCommand{name: "x", body: "{{CWD}}"}
    out, err := cmd.render(&CommandContext{})
    require.NoError(t, err)
    cwd, _ := os.Getwd()
    assert.Equal(t, cwd, out)
}

func TestSubstitute_EnvVar(t *testing.T) {
    t.Setenv("F09_TEST_VAR", "ok-value")
    cmd := &MarkdownCommand{name: "x", body:
        "{{ENV.F09_TEST_VAR}}"}
    out, err := cmd.render(&CommandContext{})
    require.NoError(t, err)
    assert.Equal(t, "ok-value", out)
}

func TestSubstitute_EnvVar_Unset(t *testing.T) {
    os.Unsetenv("F09_THIS_IS_NOT_SET")
    cmd := &MarkdownCommand{name: "x", body:
        "[{{ENV.F09_THIS_IS_NOT_SET}}]"}
    out, err := cmd.render(&CommandContext{})
    require.NoError(t, err)
    assert.Equal(t, "[]", out)
}

func TestSubstitute_FileToken_Exists(t *testing.T) {
    dir := t.TempDir()
    path := filepath.Join(dir, "include.txt")
    require.NoError(t, os.WriteFile(path, []byte("inserted-
        content"), 0644))
    cmd := &MarkdownCommand{name: "x", body: "[{{FILE:" + path +
        "}}]"}
    out, err := cmd.render(&CommandContext{})

```

```

        require.NoError(t, err)
        assert.Equal(t, "[inserted-content]", out)
    }

    func TestSubstitute_FileToken_Missing(t *testing.T) {
        cmd := &MarkdownCommand{name: "x", body: "{{FILE:/tmp/this-
            does-not-exist-12345}}"}
        out, err := cmd.render(&CommandContext{})
        require.NoError(t, err)
        assert.Contains(t, out, "FILE NOT FOUND")
    }

    func TestSubstitute_OutOfBoundsArg_EmptyString(t *testing.T) {
        cmd := &MarkdownCommand{name: "x", body: "{{ARG1}}-{{ARG2}}-
            {{ARG3}}"}
        out, err := cmd.render(&CommandContext{Args: []string{"a"}})
        require.NoError(t, err)
        assert.Equal(t, "a--", out)
    }

    func TestMarkdownCommand_ImplementsInterface(t *testing.T) {
        var _ Command = (*MarkdownCommand)(nil)
    }

    func TestMarkdownCommand_Execute(t *testing.T) {
        cmd := &MarkdownCommand{name: "x", description: "test",
            body: "Hi {{ARG1}}"}
        res, err := cmd.Execute(context.Background(),
            &CommandContext{Args: []string{"there"}})
        require.NoError(t, err)
        assert.True(t, res.Success)
        assert.Equal(t, "Hi there", strings.TrimSpace(res.Output))
    }

```

Implementation:

```

package commands

import (
    "context"
    "fmt"
    "os"
    "regexp"
    "strconv"
    "strings"

    "gopkg.in/yaml.v3"
)

```

```

// MarkdownCommand is a user-defined slash command parsed from a
//   Markdown file
// with optional YAML frontmatter.
type MarkdownCommand struct {
    name          string
    title          string
    description    string
    body          string
    variables      map[string]string
    sourcePath    string
}

func (c *MarkdownCommand) Name() string      { return c.name }
func (c *MarkdownCommand) Aliases() []string { return nil }
func (c *MarkdownCommand) Description() string { return
    c.description }
func (c *MarkdownCommand) Usage() string      { return "/" +
    c.name + " [args]" }
func (c *MarkdownCommand) SourcePath() string { return
    c.sourcePath }

func (c *MarkdownCommand) Execute(ctx context.Context, cc
    *CommandContext) (*CommandResult, error) {
    if cc == nil {
        cc = &CommandContext{}
    }
    out, err := c.render(cc)
    if err != nil {
        return nil, err
    }
    return &CommandResult{Success: true, Output: out}, nil
}

// frontmatter is the YAML shape of a Markdown command's
//   frontmatter block.
type frontmatter struct {
    Title          string          `yaml:"title"`
    Description    string          `yaml:"description"`
    Variables      map[string]string `yaml:"variables"`
}

// parseMarkdownCommand parses raw .md content into a
//   MarkdownCommand.
// Frontmatter is optional; when present it must be valid YAML.
func parseMarkdownCommand(name, raw, sourcePath string)
    (*MarkdownCommand, error) {
    cmd := &MarkdownCommand{name: name, sourcePath: sourcePath,
        variables: map[string]string{}}

```

```

body := raw
if strings.HasPrefix(raw, "---\n") {
    end := strings.Index(raw[4:], "\n---")
    if end == -1 {
        return nil, fmt.Errorf("markdown command %s:
        unterminated frontmatter", name)
    }
    fm := raw[4 : 4+end]
    body = strings.TrimPrefix(raw[4+end+4:], "\n")
    var meta frontmatter
    if err := yaml.Unmarshal([]byte(fm), &meta); err != nil {
        return nil, fmt.Errorf("markdown command %s: parse
        frontmatter: %w", name, err)
    }
    cmd.title = meta.Title
    cmd.description = meta.Description
    if meta.Variables != nil {
        cmd.variables = meta.Variables
    }
}
cmd.body = strings.TrimSpace(body)
return cmd, nil
}

// substRegex matches {{TOKEN}} forms. The token name part allows
// letters,
// digits and underscores; an optional `.suffix` (named lookup)
// and an optional
// `:argument` (file path or other inline argument) are captured
// greedily.
var substRegex = regexp.MustCompile(`\{\{([A-Z_][A-Z0-9_]*(?:\.
[A-Za-z_][A-Za-z0-9_]*)(?:[:[^\}]+)?)\}\}`)

func (c *MarkdownCommand) render(cc *CommandContext) (string,
error) {
    r := c.tokenResolver(cc)
    out := substRegex.ReplaceAllStringFunc(c.body, func(match
string) string {
        token := match[2 : len(match)-2] // strip {{ and }}
        return r(token)
    })
    return out, nil
}

func (c *MarkdownCommand) tokenResolver(cc *CommandContext)
func(token string) string {
    cwd, _ := os.Getwd()
    return func(token string) string {

```



```

if strings.HasPrefix(token, "ARG") {
    if rest := strings.TrimPrefix(token, "ARG"); rest !=
token {
        if strings.HasPrefix(rest, ".") {
            name := strings.TrimPrefix(rest, ".")
            return c.variables[name]
        }
        if n, err := strconv.Atoi(rest); err == nil && n
>= 1 {
            if n-1 < len(cc.Args) {
                return cc.Args[n-1]
            }
            return ""
        }
    }
}
if token == "SELECTION" {
    return cc.Selection
}
if token == "CURRENT_FILE" {
    return cc.CurrentFile
}
if token == "CWD" {
    return cwd
}
if strings.HasPrefix(token, "ENV.") {
    name := strings.TrimPrefix(token, "ENV.")
    return os.Getenv(name)
}
if strings.HasPrefix(token, "FILE:") {
    path := strings.TrimPrefix(token, "FILE:")
    info, err := os.Stat(path)
    if err != nil {
        return fmt.Sprintf("[FILE NOT FOUND: %s]", path)
    }
    if info.Size() > 1<<20 {
        return fmt.Sprintf("[FILE TOO LARGE: %s]", path)
    }
    data, err := os.ReadFile(path)
    if err != nil {
        return fmt.Sprintf("[FILE READ ERROR: %s: %v]",
path, err)
    }
    return string(data)
}
return "{{" + token + "}}" // unknown token – leave
verbatim

```

```
}  
}
```

CommandContext needs Selection and CurrentFile fields. If the existing struct doesn't have them, ADD them as optional string fields (zero-value safe). If CommandContext.Selection and CommandContext.CurrentFile already exist, no change.

Run failing test → impl → confirm pass → smoke clean → commit. Subject: feat(P1-F09-T02): MarkdownCommand + frontmatter parser + regex variable substitution.

Task 3: MarkdownLoader — scan dirs, register/unregister (TDD)

Files: modify internal/commands/markdown_commands.go (append MarkdownLoader types/methods), extend markdown_commands_test.go with loader tests.

Loader test cases:

```
func TestMarkdownLoader_LoadProjectAndUser(t *testing.T) {  
    projectDir := t.TempDir()  
    userDir := t.TempDir()  
    projCmds := filepath.Join(projectDir, ".helix", "commands")  
    userCmds := filepath.Join(userDir, ".config", "helixcode",  
        "commands")  
    require.NoError(t, os.MkdirAll(projCmds, 0755))  
    require.NoError(t, os.MkdirAll(userCmds, 0755))  
    require.NoError(t, os.WriteFile(filepath.Join(userCmds,  
        "shared.md"),  
        []byte("---\ndescription: from user\n---\n\nuser body"),  
        0644))  
    require.NoError(t, os.WriteFile(filepath.Join(projCmds,  
        "shared.md"),  
        []byte("---\ndescription: from project\n---\n\nproject  
body"), 0644))  
    require.NoError(t, os.WriteFile(filepath.Join(userCmds,  
        "user-only.md"),  
        []byte("only user"), 0644))  
  
    reg := NewRegistry()  
    loader := NewMarkdownLoader(reg, projCmds, userCmds)  
    require.NoError(t, loader.Load())  
  
    // "shared" registers from project (overrides user)  
    cmd, ok := reg.Get("shared")  
    require.True(t, ok)  
    mc := cmd.(*MarkdownCommand)
```

```

assert.Equal(t, "from project", mc.Description())

// "user-only" still loaded
_, ok = reg.Get("user-only")
assert.True(t, ok)
}

func TestMarkdownLoader_ReloadDiff_AddsRemovesUpdates(t
    *testing.T) {
    projectDir := t.TempDir()
    cmds := filepath.Join(projectDir, ".helix", "commands")
    require.NoError(t, os.MkdirAll(cmds, 0755))

    reg := NewRegistry()
    loader := NewMarkdownLoader(reg, cmds, "")
    require.NoError(t, loader.Load())
    _, ok := reg.Get("a")
    assert.False(t, ok)

    // Add file
    require.NoError(t, os.WriteFile(filepath.Join(cmds, "a.md"),
        []byte("body a"), 0644))
    require.NoError(t, loader.Reload())
    _, ok = reg.Get("a")
    assert.True(t, ok)

    // Remove file
    require.NoError(t, os.Remove(filepath.Join(cmds, "a.md")))
    require.NoError(t, loader.Reload())
    _, ok = reg.Get("a")
    assert.False(t, ok)
}

func TestMarkdownLoader_BadFrontmatterIsLoggedNotFatal(t
    *testing.T) {
    projectDir := t.TempDir()
    cmds := filepath.Join(projectDir, ".helix", "commands")
    require.NoError(t, os.MkdirAll(cmds, 0755))
    require.NoError(t, os.WriteFile(filepath.Join(cmds,
        "good.md"), []byte("good body"), 0644))
    require.NoError(t, os.WriteFile(filepath.Join(cmds,
        "bad.md"),
        []byte("---\nNOT YAML\n---\nbody"), 0644))

    reg := NewRegistry()
    loader := NewMarkdownLoader(reg, cmds, "")
    require.NoError(t, loader.Load()) // Load succeeds despite
    bad file

```

```

_, ok := reg.Get("good")
assert.True(t, ok)
_, ok = reg.Get("bad")
assert.False(t, ok, "bad.md must be skipped")
}

func TestMarkdownLoader_NonExistentDirsAreSkipped(t *testing.T) {
    reg := NewRegistry()
    loader := NewMarkdownLoader(reg, "/tmp/does/not/exist", "/
        tmp/also/does/not/exist")
    require.NoError(t, loader.Load())
}

```

Implementation (append to `markdown_commands.go`):

```

import (
    "path/filepath"
    "sync"

    "go.uber.org/zap"
)

// MarkdownLoader scans project + user command directories and
// registers each
// .md file as a MarkdownCommand in the supplied Registry.
// Project files
// override user files of the same name.
type MarkdownLoader struct {
    registry    *Registry
    projectDir  string
    userDir     string
    mu          sync.Mutex
    loaded      map[string]string // command name → source path
    log         *zap.Logger
}

// NewMarkdownLoader constructs a loader. projectDir and/or
// userDir may be
// empty or nonexistent; the loader gracefully handles either.
func NewMarkdownLoader(registry *Registry, projectDir, userDir
    string) *MarkdownLoader {
    return &MarkdownLoader{
        registry:    registry,
        projectDir:  projectDir,
        userDir:     userDir,
        loaded:      map[string]string{},
        log:         zap.NewNop(),
    }
}

```

```

// SetLogger replaces the no-op logger.
func (l *MarkdownLoader) SetLogger(log *zap.Logger) { l.log =
    log }

// Load scans both directories and registers each command.
// Project files win
// on name collision. Errors in individual files are logged and
// skipped, not
// fatal.
func (l *MarkdownLoader) Load() error {
    return l.Reload()
}

// Reload re-scans both directories and reconciles the registry:
// added files
// are registered, removed files are unregistered, changed files
// are replaced.
func (l *MarkdownLoader) Reload() error {
    l.mu.Lock()
    defer l.mu.Unlock()

    want := map[string]*MarkdownCommand{}
    // Order matters: user first, then project (so project
    // overrides user).
    for _, dir := range []string{l.userDir, l.projectDir} {
        if dir == "" {
            continue
        }
        entries, err := os.ReadDir(dir)
        if err != nil {
            if os.IsNotExist(err) {
                continue
            }
            l.log.Warn("markdown loader: read dir",
                zap.String("dir", dir), zap.Error(err))
            continue
        }
        for _, entry := range entries {
            if entry.IsDir() || !strings.HasSuffix(entry.Name(),
                ".md") {
                continue
            }
            name := strings.TrimSuffix(entry.Name(), ".md")
            path := filepath.Join(dir, entry.Name())
            data, err := os.ReadFile(path)
            if err != nil {

```

```

        l.log.Warn("markdown loader: read file",
zap.String("path", path), zap.Error(err))
        continue
    }
    cmd, err := parseMarkdownCommand(name, string(data),
path)
    if err != nil {
        l.log.Warn("markdown loader: parse error",
zap.String("path", path), zap.Error(err))
        continue
    }
    want[name] = cmd
}
}

// Remove names that disappeared
for name := range l.loaded {
    if _, ok := want[name]; !ok {
        l.registry.Unregister(name)
        delete(l.loaded, name)
    }
}

// Add or replace
for name, cmd := range want {
    if existing, ok := l.registry.Get(name); ok {
        if _, isMd := existing.(*MarkdownCommand); !isMd {
            // Don't overwrite a built-in command with a
            Markdown one.
            l.log.Warn("markdown loader: name conflicts with
built-in",
                zap.String("name", name),
zap.String("source", cmd.sourcePath))
            continue
        }
        l.registry.Unregister(name)
    }
    if err := l.registry.Register(cmd); err != nil {
        l.log.Warn("markdown loader: register",
zap.String("name", name), zap.Error(err))
        continue
    }
    l.loaded[name] = cmd.sourcePath
}
return nil
}

// Loaded returns a snapshot of name → source path.
func (l *MarkdownLoader) Loaded() map[string]string {

```

```

    l.mu.Lock()
    defer l.mu.Unlock()
    out := make(map[string]string, len(l.loaded))
    for k, v := range l.loaded {
        out[k] = v
    }
    return out
}

```

Registry.Unregister(name string) may not exist yet — add it (1-line method that takes mu.Lock and deletes from the map).

Run TDD cycle. Subject: feat(P1-F09-T03): MarkdownLoader scans project + user dirs, registers/unregisters Markdown commands.

Task 4: markdown_watcher.go — fsnotify + 200ms debounce (TDD)

Files: create helix_code/internal/commands/markdown_watcher.go, helix_code/internal/commands/markdown_watcher_test.go.

Test cases:

```

func TestWatcher_DebouncesAndReloads(t *testing.T) {
    dir := t.TempDir()
    cmds := filepath.Join(dir, ".helix", "commands")
    require.NoError(t, os.MkdirAll(cmds, 0755))

    reg := NewRegistry()
    loader := NewMarkdownLoader(reg, cmds, "")
    require.NoError(t, loader.Load())

    w, err := NewMarkdownWatcher(loader, []string{cmds})
    require.NoError(t, err)
    w.SetDebounce(50 * time.Millisecond)
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()
    go w.Run(ctx)
    time.Sleep(50 * time.Millisecond) // let watcher subscribe

    // Write a file; expect to be loaded after debounce
    require.NoError(t, os.WriteFile(filepath.Join(cmds,
        "added.md"), []byte("hi"), 0644))
    require.Eventually(t, func() bool {
        _, ok := reg.Get("added")
        return ok
    }, 2*time.Second, 25*time.Millisecond)
}

```

```

func TestWatcher_StopsOnContextCancel(t *testing.T) {
    dir := t.TempDir()
    cmds := filepath.Join(dir, ".helix", "commands")
    require.NoError(t, os.MkdirAll(cmds, 0755))
    reg := NewRegistry()
    loader := NewMarkdownLoader(reg, cmds, "")
    w, err := NewMarkdownWatcher(loader, []string{cmds})
    require.NoError(t, err)
    ctx, cancel := context.WithCancel(context.Background())
    done := make(chan struct{})
    go func() { w.Run(ctx); close(done) }()
    cancel()
    select {
    case <-done:
    case <-time.After(2 * time.Second):
        t.Fatal("watcher did not stop on ctx cancel")
    }
}

func TestWatcher_HandlesMissingDirInitially(t *testing.T) {
    w, err := NewMarkdownWatcher(nil, []string{"/tmp/does/not/exist"})
    if err != nil {
        t.Skip("fsnotify cannot watch missing dir on this platform – acceptable")
    }
    defer w.Close()
}

```

Implementation:

```

package commands

import (
    "context"
    "sync"
    "time"

    "github.com/fsnotify/fsnotify"
    "go.uber.org/zap"
)

// MarkdownWatcher uses fsnotify to detect changes in command
// directories and
// triggers loader.Reload() with a debounce so rapid filesystem
// activity
// (editor saves) collapses to a single reload.
type MarkdownWatcher struct {

```



```

    loader    *MarkdownLoader
    dirs      []string
    debounce  time.Duration
    w         *fsnotify.Watcher
    log       *zap.Logger
    mu        sync.Mutex
    pending   *time.Timer
}

func NewMarkdownWatcher(loader *MarkdownLoader, dirs []string)
    (*MarkdownWatcher, error) {
    w, err := fsnotify.NewWatcher()
    if err != nil {
        return nil, err
    }
    for _, d := range dirs {
        if d == "" {
            continue
        }
        if err := w.Add(d); err != nil {
            // Non-fatal: directory may not exist yet. Log via
            // no-op logger.
        }
    }
    return &MarkdownWatcher{
        loader:    loader,
        dirs:      dirs,
        debounce:  200 * time.Millisecond,
        w:         w,
        log:       zap.NewNop(),
    }, nil
}

func (w *MarkdownWatcher) SetDebounce(d time.Duration) {
    w.debounce = d
}
func (w *MarkdownWatcher) SetLogger(l *zap.Logger)      { w.log = l }
func (w *MarkdownWatcher) Close() error                { return w.w.Close() }

// Run blocks until ctx is cancelled, processing watcher events
// with a
// debounced Reload.
func (w *MarkdownWatcher) Run(ctx context.Context) {
    defer w.w.Close()
    for {
        select {
        case <-ctx.Done():

```

```

        return
    case ev, ok := <-w.w.Events:
        if !ok {
            return
        }
        if ev.Op == 0 {
            continue
        }
        w.scheduleReload()
    case err, ok := <-w.w.Errors:
        if !ok {
            return
        }
        w.log.Warn("markdown watcher: error", zap.Error(err))
    }
}

func (w *MarkdownWatcher) scheduleReload() {
    w.mu.Lock()
    defer w.mu.Unlock()
    if w.pending != nil {
        w.pending.Stop()
    }
    w.pending = time.AfterFunc(w.debounce, func() {
        if w.loader == nil {
            return
        }
        if err := w.loader.Reload(); err != nil {
            w.log.Warn("markdown watcher: reload",
                zap.Error(err))
        }
    })
}

```

Run TDD cycle. Subject: feat(P1-F09-T04): markdown_watcher.go fsnotify + debounced reload.

Task 5: /commands slash + helixcode commands cobra (TDD)

Files: create helix_code/internal/commands/commands_command.go, helix_code/internal/commands/commands_command_test.go, helix_code/cmd/cli/commands_cmd.go, helix_code/cmd/cli/commands_cmd_test.go.

/commands slash: - /commands (default list) → table NAME / TITLE / SOURCE - /commands show <name> → renders body of that command (without args) - /commands reload → calls loader.Reload(); reports added/removed - /commands run <name>

<args...> → calls cmd.Execute(ctx, &CommandContext{Args: args}); prints output -
Unknown subcommand → error

helixcode commands cobra: - helixcode commands list (mirror of slash list, table)
- helixcode commands show <name> - helixcode commands reload (no-op for one-shot CLI; informational) - helixcode commands run <name> [args...]

Both surfaces use the same loader. Cobra-side construction in commands_cmd.go follows the pattern from F06 mcp_cmd.go.

Run TDD cycle. Subject: feat(P1-F09-T05): /commands slash + helixcode commands cobra (list/show/reload/run).

Task 6: cmd/cli/main.go startup wiring + integration test

Add to main.go after F08 wiring:

```
projectCmds := filepath.Join(".", ".helix", "commands")
var userCmds string
if home, err := os.UserConfigDir(); err == nil {
    userCmds = filepath.Join(home, "helixcode", "commands")
}
mdLoader := commands.NewMarkdownLoader(cmdRegistry, projectCmds,
    userCmds)
mdLoader.SetLogger(logger)
if err := mdLoader.Load(); err != nil {
    log.Printf("markdown commands: load failed: %v", err)
}
mdWatcher, err := commands.NewMarkdownWatcher(mdLoader,
    []string{projectCmds, userCmds})
if err == nil {
    mdWatcher.SetLogger(logger)
    go mdWatcher.Run(ctx)
    defer mdWatcher.Close()
}
if err :=
    cmdRegistry.Register(commands.NewCommandsCommand(mdLoader));
    err != nil {
    log.Printf("commands: register slash failed: %v", err)
}
rootCmd.AddCommand(newCommandsCmd(commandsCmdDeps{Loader:
    mdLoader, Registry: cmdRegistry}))
```

Integration test tests/integration/markdown_commands_test.go (//go:build integration): - Create temp project dir with .helix/commands/echo.md body Got: {{ARG1}} - Construct loader, run command via Execute, assert output Got: hello world - Write a second file via watcher, assert it appears in registry within 1s

Subject: feat(P1-F09-T06): wire markdown loader + watcher into main.go + integration test.

Task 7: Challenge with runtime evidence

Harness `helix_code/tests/integration/cmd/p1f09_challenge/main.go`: 1. Create temp `.helix/commands/echo.md` with `Got: {{ARG1}}` 2. Build registry, load, run echo with arg "hello world" 3. Assert output contains `Got: hello world` 4. Update file body to `New: {{ARG1}}`, trigger reload, run again 5. Assert output contains `New: hello world`

`run.sh`, `CHALLENGE.md`, evidence in `06_phase_1_evidence.md`. Dual commit (submodule + meta-repo).

Subject: `feat(P1-F09-T07): challenge with runtime evidence + cross-compile check`.

Task 8: Feature 9 close-out + push to 4 remotes

Tick all 8 P1-F09 task list items. Update PROGRESS current focus to idle (F10 candidate). Final unit + integration + smoke + cross-compile + go vet + challenge re-run. Commit chore (P1-F09-T08): Feature 9 (Slash Command System) close-out. Push origin/github/gitlab/upstream non-force (CONST-043).

Self-review notes

1. Spec coverage: every spec section has a task — `types/parser/substitution` (T02), `loader` (T03), `watcher` (T04), `surfaces` (T05), `wiring + integration` (T06), `challenge` (T07), `close-out` (T08).
2. TDD: every code task starts with failing tests.
3. Type consistency: `MarkdownCommand`, `MarkdownLoader`, `MarkdownWatcher`, `parseMarkdownCommand`, `tokenResolver`, `substRegex`, `Registry.Unregister`, `CommandContext.Selection`, `CommandContext.CurrentFile` — all used consistently.
4. Cross-platform: `fsnotify` works on Linux+Windows; cross-compile linux is the canonical check.
5. Anti-bluff: full 4-term smoke + Challenge captures real rendered output from a real `.md` file.
6. No new external deps — `fsnotify`, `yaml.v3`, `cobra`, `testify` all in `go.mod`.
7. Branch + push: stays on main, non-force to all four remotes.